

Exact method and Genetic algorithm for the symmetric Travelling Salesperson Problem

in partial fulfillment for the "Methods and Models for Combinatorial Optimization" course unit

UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

MASTER'S DEGREE IN COMPUTER SCIENCE

Luca Veronese

mat. 2089404

`luca.veronese.5@studenti.unipd.it`

February 12, 2025

1 Problem Definition and Approach

In this project, we address the problem of minimizing non-productive time during the drilling process, that consists in relocating the drill bit to a new position in order to drill another hole.

We develop two solution approaches:

- **Exact approach:** We formulated the problem as a mixed integer-linear programming (MILP) model and solved it using IBM CPLEX to obtain optimal solutions.
- **Heuristic approach:** We also designed a genetic algorithm (GA) implemented in C++. This approach aims to find an approximate solution within a smaller computational time compared to the exact approach.

Assumptions: We assume that the drilling sequence is predefined and all hole locations are known in advance. We further assume that the drill can move at constant speed in any direction, and that all holes are of the same size and can be drilled with the same drill bit. The repositioning time is determined solely by the (euclidean) travel distance of the drill bit.

Notice that the constant speed is an overly simplifying assumption for the real-world. The acceleration and deceleration of the tool are significant factors, especially when dealing with short distances [KHODABAKHSHI2021426].

2 Exact method

2.1 Mixed Integer-Linear Programming formulation

The following is the MILP formulation that has been implemented in order to solve exactly the symmetric TSP problem. It is inspired by formulation in [Gavish1978TheTS].

Sets:

- N : set of graph nodes, representing the holes.

- A : set of arcs (i, j) for all $i, j \in N$, representing the trajectory covered by the drill to move from hole i to hole j .

Parameters:

- c_{ij} : time taken by the drill to move from i to j , for all $(i, j) \in A$;
- 0 : arbitrarily selected starting node, where $0 \in N$.

Decision Variables:

- x_{ij} : amount of the flow shipped from i to j , for all $(i, j) \in A$;
- y_{ij} : 1 if arc (i, j) ships some flow, 0 otherwise, for all $(i, j) \in A$.

Formulation:

$$\min \sum_{i,j:(i,j) \in A} c_{ij} y_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{i:(i,k) \in A} x_{ik} - \sum_{j:(k,j) \in A, j \neq 0} x_{kj} = 1 \quad \forall k \in N \quad (2)$$

$$\sum_{j:(i,j) \in A} y_{ij} = 1 \quad \forall i \in N \quad (3)$$

$$\sum_{i:(i,j) \in A} y_{ij} = 1 \quad \forall j \in N \quad (4)$$

$$x_{ij} \leq (|N| - 1) y_{ij} \quad \forall (i, j) \in A, j \neq 0 \quad (5)$$

$$x_{ij} \in \mathbb{R} \quad \forall (i, j) \in A, j \neq 0 \quad (6)$$

$$y_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (7)$$

2.2 Implementation

Variables and objective function Let n be the number of nodes. The number e of undirected edges (excluding self-loops) is equivalent to $n * (n - 1)$. For our problem we declared e binary variables $y_0, \dots, y_{e-1}$ and $n * (n - 1)$ integer variables, $x_0, \dots, x_{e-1}$. Notice that every variable represent a directed arc that is not a self-loop, and viceversa every directed arc that is not a self-loop is represented by a variable x and a variable y .¹

The objective function is a sum of e variables, precisely $\sum_{i \in [0, e) \cup \mathbb{N}} y_i$, and our goal is to minimize it.

Rows We had to rewrite constraint 5 so to have a real number on the right-hand side:

$$x_{ij} - (|N| - 1) y_{ij} \leq 0 \quad \forall (i, j) \in A, j \neq 0$$

Let $\mathbf{u}$ be the vector $[y_0, \dots, y_{e-1}, x_0, \dots, x_{e-1}]$. Let F, G, H, L be matrices such that constraints 2–5 are equivalent to $F\mathbf{u} = 1, G\mathbf{u} = 1, H\mathbf{u} = 1, L\mathbf{u} \leq 0$

For every $\eta \in \{F, G, H, L\}$ we have defined the arrays ηsense , ηmatbeg , ηmatind , ηmatval , $\eta\text{rowname}$ and the integers ηrcnt and ηnzcnt to be passed to `CPXaddrows`. In this way, only four (checked) calls to `CPXaddrows` are performed to solve the mixed integer-linear problem.

¹In facts the class `Graph` defines a one-to-one mapping between integers $k \in [0, e) \cup \mathbb{N}$ and couples of integer (i, j) s.t. $i, j < n \wedge i \neq j$

Classes The program is contained almost entirely within the `main()` function. This style follows that of CPLEX library examples, which are typically written inside the `main()` procedure, and has been implemented because of the limited size of the codebase in term of single lines of code (*s.l.o.c.*) and the absence of need to separate logic into multiple classes.

Few additional classes are used:

- **TSPGraphReader**, responsible for reading and parsing a graph from file.
- **Graph**, which stores a the adjacency matrix of a graph and provides static method to convert from single coordinate to double coordinate.
- **Node**, that represents a two-dimensional point, to be used as a vertex of a **Graph**. Provides a method that returns the distance from another **Node**.
- **Path**, that represents an acyclic path in the graph.
- **CommandLineOptions**, which handles command-line arguments such as the input filename.

The header `cpxmacro.h`, that defines the macros `DECL_ENV`, `DECL_PROB`, and `CHECKED_CPX_CALL`, the array of character `errmsg` and the integer `status` has been included in our project.

We have used Valgrind Static Analyzer to check for memory leaks.

2.3 Handling the MIP Optimality Gap in CPLEX

In our experiments, for many problem instances, regardless of the computation time allocated to the solver, the termination status returned by CPLEX is `CPXMIP_OPTIMAL_TOL`, which indicates that an optimal solution has been found within the tolerance defined by either the relative or absolute MIP gap.

In solving mixed-integer programming (MIP) problems using CPLEX, one common approach to achieving near-exact optimality is to explicitly tighten the optimality gap by setting the parameter `CPXPARAM_MIP_Tolerances_MIPGap` to a sufficiently small value (e.g., 10^{-6}).

However, in our experiments, setting `CPXPARAM_MIP_Tolerances_MIPGap` to very small values resulted in memory corruption errors, such as `double free or corruption (!prev)`. Given such failures, manually tightening the MIP optimality gap was deemed infeasible. Hence the default value for `CPXPARAM_MIP_Tolerances_MIPGap` (i.e. 10^{-3}) has been left unchanged.

3 Genetic algorithm

3.1 Outline

For the second assignment, a **genetic algorithm** has been designed to solve the Traveling Salesperson Problem (TSP).

A *graph* is represented using an adjacency matrix, and an hamiltonian *tour* is expressed as a vector of non-repeating integer numbers, each corresponding to a node. The length of the vector corresponding to a tour matches the number of nodes in the graph. It is worthy to note that any hamiltonian tour can be expressed in many ways as a vectors of node, since a cycle has no starting node.

The hamiltonian tour is considered as the individual in our problem, and the fitness function is the inverse total weight of the edges traversed in the tour. Each individual is a valid possible solution for our problem. Although we are aware that is not a compulsory characteristic, this feature avoids us the process of "fixing" an unfeasible solution.

Various selection, crossover, and mutation operators have been implemented within a common algorithmic framework, including:

- Tournament selection
- Ranking selection
- Order crossover (OX)
- Partially mapped crossover (PMX)
- Reverse Sequence Mutation (PSM)
- Partial Shuffle Mutation (RSM)

3.2 Initial population generation

The initial population consists of randomly generated permutation of nodes, each having as many node as the graph of the problem.

Few individuals (5%) are generated using one of the following heuristics.

3.2.1 Nearest neighbor heuristics

Begin at a random node and build a tour by always moving to the nearest neighbor (among the nodes not yet connected to the tour) of the latest node to join the tour. When all nodes have been connected to the tour, join the last node to the origin to complete the tour. **[mitlogisticaltransportation]**

Worst-case performance: $\frac{1}{2} \lceil \log_2 n \rceil + \frac{1}{2}$

Complexity: $O(n^2)$.

3.2.2 Farthest insertion

The tour construction process begins with an initial loop consisting of only two nodes. At each iteration, a new node is inserted into the tour.

The insertion cost is defined as:

$$c(i, k, j) = d(i, k) + d(k, j) - d(i, j), \quad (8)$$

where i and j are consecutive nodes in the current tour T , and $d(a, b)$ represents the distance between nodes a and b .

The next node to be added to the tour is the one that maximizes $d(k, T)$, i.e., the node farthest from the existing tour. Once selected, the node is inserted at the position (i, j) that minimizes the insertion cost $c(i, k, j)$. This process is repeated iteratively until all nodes have been incorporated into the tour T .

Worst-case performance: unknown **[mitlogisticaltransportation]**

Complexity: $O(n^2)$.

3.3 Crossover operators

Both the following crossover have been implemented.

3.3.1 Partially Mapped Crossover (PMX)

Partially Mapped Crossover (PMX) is a recombination operator used in permutation-based genetic algorithms, particularly for problems such as the Traveling Salesman Problem (TSP). It ensures that offspring maintain valid permutations while inheriting information from both parents.

Let P_0 and P_1 be the first and second parent chromosomes. The algorithm is the following [crossover-wikipedia]

1. Randomly select two crossover points in P_0 .
2. Copy the segment between these two points from P_0 to the child chromosome at the same position.
3. Identify genes that have not been copied by checking the corresponding segment in P_1 , starting from the first crossover point.
4. For each missing gene (denoted as m):
 - Find the gene n in the offspring that was copied from P_0 in its place.
 - Copy m to the position held by n in P_1 if that position is unoccupied.
 - Otherwise, move to the next missing gene and repeat the process.

This method preserves positional information and maintains valid permutations.

3.3.2 Order Crossover (OX)

Order Crossover (OX) is a recombination operator used in permutation-based genetic algorithms. It ensures that the offspring inherit the relative order of elements from both parents.

Let P_0 and P_1 be the first and second parent chromosomes. The algorithm is the following [crossover-wikipedia]:

1. Randomly select two crossover points in P_0 .
2. Copy the segment between these points from P_0 to the child chromosome at the same position.
3. Traverse P_1 from the second crossover point onward (wrapping around if necessary) and copy the remaining genes in the same order as they appear in P_1 , skipping any genes already present in the child.

This method preserves the relative order of genes while maintaining valid permutations in the offspring.

3.4 Genetic operators

Both the following crossover have been implemented.

3.4.1 Reverse Sequence Mutation (RSM)

The Reverse Sequence Mutation [DBLP:journals/corr/abs-1203-3099] reverses a sequence limited by two position i and j such that $0 \leq i < j \leq n - 1$, where n is the chromosome length (in our case the number of holes).

3.4.2 Partial Shuffle Mutation (PSM)

The Partial Shuffle Mutation [DBLP:journals/corr/abs-1203-3099] swap one or more couple of elements in the chromosome. In our implementation a PSM mutation swaps only a couple of element.

3.5 Selection operators

We have implemented two selection operators: the linear ranking selection operator and the k -tournament selection operator.

The linear ranking selection operator assigns a rank to each individual in the population based on their fitness values, with higher-ranked individuals having a greater probability of being selected for reproduction. The k -tournament selection operator involves randomly selecting a subset of individuals from the population and choosing the best among them as a parent.

3.6 Survival selection

We choosed the (μ, γ) evolution strategy. In each iteration, λ offspring are generated, from which the $\mu < \lambda$ best individuals are selected to form the next generation. Following [oslobioinspired], we set the offspring size as $\lambda = 3\mu$, which provides a good balance between exploration and selection pressure.

The offspring selection is based on the fitness rankings, not on the actual fitness values. The resulting algorithm is therefore invariant with respect to monotonic transformations of the objective function.

An alternative evolution strategy is $(\mu + \gamma)$, in which the best are selected from a union of μ parents and λ offspring. The strategy is considered worse than (μ, γ) in leaving local optima [oslobioinspired].

3.7 Elitism

We let the user specify an elitism rate in $[0,1]$, which preserves the best solutions across generations to prevent losing high-quality candidates. Specifically, given the *elitism_rate* we retain *elitism_counter* (computed as $\mu * \text{elitism_rate}$) best parents and select the best $(\mu - \text{elitism_counter})$ offspring to form the next generation's population of size μ .

4 Implementation

4.1 Class Design

The following classes have been defined. Each class has been designed with minimal responsibilities, adhering to the principle of single responsibility: This explain the relatively large number of classes in the project.

Description of trivial methods such as constructors and conversion operators are omitted.

- The **Node** class represent a two-dimensional point, to be used as a vertex of a **Graph**. Contains a method `double distance(const Node&) const`, that returns the distance from another **Node**.

- The **Graph** class represent a complete weighted graph. A **Graph** is constructed from an array of **Node**, representing its vertices. The weights of the graph's edges are stored in a (private) two-dimensional vector of **double** values.

- The **Tour** class represents a single candidate solution and is responsible for storing and manipulating a chromosome.

It contains a vector for the sequence of node of a hamiltonian tour, and a double representing the tour's cost. Two methods are provided to reverse a sub-path (**reverse_sub_path(int from, int to)**) and to swap two indices (**swap_indexes(int i, int j)**). The tour's cost is updated to reflect the current state of the tour after construction and any operations.

- the **FitnessEvaluator** class calculates the cost of a **Tour** based on the total weight of the edges traversed in the order specified by the **Tour** object. At **FitnessEvaluator** is constructed after a **Graph** object, which contains the adjacency matrix that is necessary information to compute the cost value.

- The **Solver** class represents the core of the genetic algorithm process. The method **solve** define the main loop (where the evolution take place) and its termination condition. The loop makes polymorphic calls to methods of **_mutation_operator**, **_crossover_operator**, and **_selection_operator**, i.e. utilizes different genetic operator depending on which object the user passed to the constructor.

An object of class **Solver** needs the caller to set a **FitnessEvaluator**, and all the genetic operators (for Mutation, Crossover, and Selection) in order for the **solve** method to be used on an population (vector) of **Tour**

4.1.1 Genetic operators

Three *abstract* classes are defined as being the base for genetic operators:

- **MutationOperator**, having a public virtual method **mutate**, that is implemented by the two concrete subclasses, **PSMutation** and **RSMutation**.
- **CrossoverOperator** having a public virtual method **crossover**, that is implemented by the two concrete subclasses, **OrderCrossover** and **PMXCrossover**.
- **SelectionOperator** having a public virtual method **select**, that is implemented by the two concrete subclasses, **TournamentSelection** and **RankingSelection**.

Extensibility A user can define their own genetic operators by subclassing a proper class and subsequently instantiate a **Solver** that uses the newly defined operators.

Similarity with strategy pattern The design of class **Solver** and genetic operators classes is loosely inspired by the well-known strategy pattern. In our implementation, the caller can specify the strategy by providing an instance of a specific subclass to **Solver**'s constructor. However, this strategy is immutable at runtime, meaning it cannot be altered once the object has been instantiated.

4.1.2 Helper classes

- `TSPGraphReader` provides a method `read(const std::string &filepath)` that returns a vector of `Node` corresponding to point indicated by a valid file in TSPLIB-format. The TSPLIB-format is the following:

```
%..some headers..  
NODE_COORD_SECTION  
1   x1   y1  
2   x2   y2  
%..some other points  
EOF
```

- `CommandLineOptions` is responsible for parsing and handling command-line arguments.

Note - Iterators and access operators

Most of our data structures provide `begin()`, `end()`, and `operator[]`. These methods:

- ensure compatibility with STL algorithms that operates on iterator pairs (such as `std::find`, `std::sort`, and `std::accumulate`);
- allow usage in constructs like:

```
for (auto it = container.begin(); it != container.end(); ++it) { ... }
```

and:

```
for (auto elem : container) { ... }
```

Furthermore, `cbegin()` and `cend()` return constant iterators, and are used to explicitly indicate that the referenced elements remain unmodified.

4.2 Usage

Available options can be displayed by running `assignment2 -h`. The output is the following:

`assignment2 [options]`

options:

<code>-f, --filename [FILE]</code>	Name of the input TSP instance file
<code>-t, --timeout-ms [TIMEOUT]</code>	Timeout expressed in milliseconds
<code>-m, --mutation-probability [PROBABILITY]</code>	Probability that a mutation occurs in the genetic algorithm
<code>-c, --crossover-rate [RATE]</code>	Probability that two selected solutions are mated to create a new offspring
<code>-e, --elitism-rate [RATE]</code>	Elitism rate
<code>--mu [SIZE]</code>	Size of the population pool

<code>--lambda [SIZE]</code>	Size of the offspring pool before being pruned
<code>-k, --tournament-size [SIZE]</code>	Size of the tournament selection
<code>-N, --max-gen-no-improvement [NUMBER]</code>	Maximum number of generations without solution improvement
<code>-M, --max-gen [NUMBER]</code>	Maximum number of generations
<code>-b, --best [BOOL]</code>	If true, runs the algorithm once with the best combination of genetic operators; if false, runs it 8 times with different combinations

Any run of `assignment1 -f FILENAME` generates:

- a file having extension `.lp` which contains the description of the model as obtained by `CPXwriteprob`;
- the solution to the problem (as an ordered sequence of integers representing the nodes), the optimal cost (as an euclidean distance) and the eventual MIP gap; in `stdout`.

Any run of `assignment1 -f FILENAME` generates, for each of the 8 combinations of genetic operator implemented, the approximate solution to the problem (as an ordered sequence of integers representing the nodes) and its cost (as an euclidean distance), in `stdout`.

5 Further improvements

To refine the solutions produced by genetic algorithm, we propose the incorporation of local optimization techniques on the permutation generated by our algorithm; for example the 2-opt algorithm or Tabu Search.

The 2-opt algorithm systematically improves the solution by iteratively removing two crossing segments and reconnecting the resulting paths, thereby reducing the overall tour length through incremental improvements: In facts 2-opt is guaranteed to never output a solution worse than the previous one.

Alternatively, Tabu Search can be used to explore the solution space more broadly. By maintaining a tabu list of recently visited solutions, Tabu Search allows for the acceptance of non-improving moves, which helps to escape local optima and encourages a more diverse search.

A different fitness function that penalizes solution similar to the current best one might be tried.

6 Tests and performance

To evaluate the performance of the algorithm, tests were conducted on a machine running Fedora Linux 40. The hardware specifications include an AMD Ryzen 5 7640U (6-core, 12-thread) processor and 32GB of DDR5 RAM (5600 MHz, dual channel).

Wall-clock time and CPU time for the exact and metaheuristic solution were measured using `/usr/bin/time`.

6.1 Random Instance Generation

To generate problem instances initially a standard distribution was utilized, resulting in the formation of dense clusters that are spatially separated from one another.

To introduce additional variation and achieve a more realistic distribution, a uniform distribution was incorporated. This addition populates the remaining space with less dense points, leading to a structure that loosely resembles distinct components (clusters) connected by a sparse network of points; hopefully similar to a printed circuit layout.

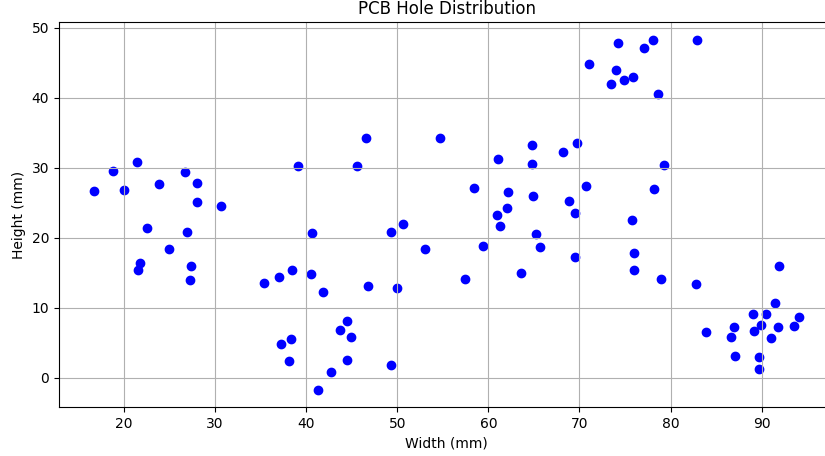


Figure 1: Random instance with 94 holes

7 Discussion

The exact method was run until the optimal solution was obtained (with possible tolerance) for instances with fewer than 100 nodes. The computational complexity appears to be highly non-linear: the solver takes approximately 10 seconds to solve a 64-node instance and 80 seconds for a 94-node instance.

The genetic algorithm is effective in improving the initial population only when this population is highly unfit initially.

In facts, the presence of even a single Nearest Neighbor solution in an otherwise random initial population creates a local optimum that not a single of our genetic algorithms can overcome, causing our algorithm to output the initial solution.

In contrast, initializing the population with 10% Farthest Insertion solutions and 90% random permutations allows the algorithm to effectively explore and improve upon the initial solutions, never reaching the quality of a solution generated with nearest neighbor anyway.